SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1708**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. *(BL3)*

Assessment Tool Weightage: 30%

QUESTIONS: 2

Duration : 45 Minutes
Total Marks:20

Annexure A

Descriptive Written Test

Code of Conduct:

I Ganesh Ram V (1924260258) certify that this submission is my original work and that
I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Question 1: Intelligent Agents in a Smart Home System (10 Marks)

1. Introduction

Anintelligent agent perceives its environment through sensors and acts on it through actuators to achieve its objectives. In a smart home, the sensors are motion detectors, temperature sensors, door/window contacts, cameras and a clock; the actuators are lights, the HVAC (heating/cooling), door locks and alarms. The environment is partially observable, dynamic, stochastic and continuous, so the choice of agent architecture strongly affects performance.

2. Types of Intelligent Agents and Their Role in the Smart Home

(a)SimpleReflexAgent. Itselectsan actionusing onlythe currentpercept through condition– action rules (IF condition THEN action). It keeps no memory or internal model.

- **Working in the system:** IF motion detected AND room is dark THEN switch on the light; IF temperature > 30 °C THEN switch on the AC; IF door opened while the system is armed THEN sound the alarm.

Strength: very fast, simple and reliable, which is ideal for safety-critical instant responses.

Limitation: it cannot handle partial observability (it cannot tell whether a person is still in a room that has no motion), cannot learn preferences and cannot plan.

(b) Model-Based Reflex Agent. It maintains an internal state that tracks how the world evolves and how its own actions affect it, so it can act sensibly even when the current percept is incomplete.

- **Working in the system:** it remembers that someone entered the living room and has not left, so it does not turn the lights off when the person sits still. It tracks which windows are open, whether the house is occupied and the room temperature trend.

- **Strength:** handles partial observability and avoids false triggers.
- **Limitation:** still reacts through rules; it has no explicit goals and does not weigh trade-offs.

(c) Goal-Based Agent. It uses the world model plus explicit goal information and chooses actions (using search or planning) that lead to the goal.

• **Working in the system:** goal “room at 24 °C by 7 PM when the user returns” makes the agent plan when to start pre-cooling; goal “house secure before sleep” makes it plan to lock doors, close windows and arm sensors in order.

Strength: flexible, since changing the goal changes behaviour without rewriting rules.

Limitation: goals are binary (achieved or not). It cannot compare two ways of reaching the same goal, such as a fast but costly way versus a slower, cheaper way.

(d) Utility-Based Agent. It assigns a utility value to each possible state and chooses the action with the maximum expected utility, balancing conflicting objectives under uncertainty.

• **Working in the system:** a utility function combines comfort, energy cost and security, for example U = w1·Comfort − w2·EnergyCost + w3·Safety. The agent might accept a temperature 1 °C off the preferred value to save energy when electricity is expensive, but never lowers security. It also learns the weights from user feedback.

Strength: makes rational trade-offs and handles uncertainty and learning.

Limitation: needs a well-designed utility function and more computation.

3. Comparison

Feature	Simple Reflex	Model-Based	Goal-Based	Utility-Based
Uses memory/model	No Yes		Yes	Yes
Uses goals	No No No No		Yes	Yes
Handles trade-offs			Limited	Yes
Speed	Fastest Fast		Moderate	Slower
Smart-home example	Motion → light ON Track occupancy		Pre-cool by 7 PM	Balance comfort vs cost

4. Most EffectiveApproach: AHybrid(Layered) Agent

No single type satisfies all requirements, so a hybrid layered architecture is the best choice:

- **Layer 1 – Reflex layer (security and safety):** simple reflex rules give instant responses to intrusion, smoke or gas alarms. Latency matters more than optimisation here, and these rules must never be overridden by cost considerations.

• **Layer 2 – Model-based layer (situation awareness):** keeps the state of occupancy, door/window status, temperature trends and time of day, which removes false alarms and wasted energy.

- **Layer3–Goal-basedlayer(planning):** plans routines such as pre-heating before arrival, night-mode and away-mode.

- **Layer 4 – Utility-based layer with learning (optimisation):** continuously balances comfort and energy efficiency and adapts to user preferences over time through learning from user overrides.

Justification: comfort is achieved by the model-based, goal-based and learning layers; energy efficiency is achieved by the utility layer, which weighs consumption against comfort; security is guaranteed by the reflex layer, which has the highest priority. The hybrid design therefore combines the speed of reflex agents with the intelligence and adaptability of utility-based agents.

Question 2: Maze Robot – Uninformed Search and Agent Design (10 Marks)

1. ProblemFormulation

- **State:** the robot's current cell (position) in the maze.
- **Initial state / Goal test:** start cell / the exit cell.
- **Actions:** move up, down, left or right (if no wall).
- **Path cost:** cost per move (1 per step, or higher for difficult cells such as turns or rough terrain).

Because the layout is unknown and no heuristic is available, the robot must use uninformed (blind) search strategies, which use only the problem definition and no domain knowledge about how close a cell is to the exit.

2. Uniform Cost Search(UCS)

UCS expands the node with the lowest path cost g(n) first, using a priority queue. It is BFS generalised to variable step costs. Each time the robot discovers new cells, it adds them to the frontier with their cumulative cost and always expands the cheapest one. When the exit is selected for expansion, the path found is guaranteed to be the least-cost path.

- **Completeness:** yes, provided every step cost is at least $\varepsilon > 0$.
- **Optimality:** yes, it returns the cheapest path.
- **Time complexity:** $O(b^{(1+\lfloor C^*/\varepsilon \rfloor)})$, where b is the branching factor, C^* the optimal cost and ε the minimum step cost.
- **Space complexity:** $O(b^{(1+\lfloor C^*/\varepsilon \rfloor)})$, because it stores all frontier nodes.
- **Advantage:** optimal even with unequal costs.
- **Disadvantage:** explores in all directions, so it can be slow and uses very large memory in big mazes; it may explore many useless cheap paths.

3. Iterative Deepening Search (IDS)

IDS runs depth-limited DFS repeatedly with limits 0, 1, 2, 3 … until the goal is found. It combines the low memory use of DFS with the completeness and shallowest-solution property of BFS. In

the maze, the robot explores to depth 1, then restarts and explores to depth 2, and so on. Re-exploring upper levels is not costly because most nodes are at the deepest level.

- **Completeness:** yes (for finite branching factor).
- **Optimality:** yes when all step costs are equal (finds the shallowest goal); not for variable costs.
- **Time complexity:** O(b^d), where d is the depth of the shallowest goal.
- **Space complexity:** O(b·d), which is linear and very small.
- **Advantage:** very memory efficient, avoids infinite-loop problems of plain DFS (with cycle checking) and finds the shallowest exit.
- **Disadvantage:** repeated expansion of nodes (about 10–11% overhead for b = 10) and it ignores step costs.

4. Comparison of UCS and IDS

Criterion	UCS	IDS
Expands by	Lowest path cost g(n)	Increasing depth limit
Complete	Yes (ε > 0)	Yes
Optimal	Yes (any positive cost)	Only for uniform step cost
Time	O(b^(1+⌊C*/ε⌋))	O(b^d)
Space	Exponential	O(b·d) – linear
Best used when	Move costs differ and memory is available	Memory is limited and costs are uniform

5. Relating the Problem to AgentTypes

- **Simple reflex agent:** rules like "if wall ahead, turn right" can easily loop forever in a maze; it has no memory of visited cells, so it is unsuitable on its own.
- **Model-based agent:** builds a map of visited cells and walls from its sensors, which overcomes partial observability and lets it avoid revisiting cells.
- **Goal-based agent:** the goal is "reach the exit". It uses search (UCS/IDS) over its model to plan a path, so this is the natural fit for the maze problem.
- **Utility-based agent:** chooses among paths using utility (shortest distance, least battery, avoiding hazards); useful when paths have different costs or risks.

Agent design affects decision-making because the architecture decides what information is used (current percept, memory, goals or utility) and therefore how good, how fast and how adaptable the decisions are. A reflex design decides instantly but myopically; goal and utility designs decide deliberately but need more computation and memory.

6. Recommended Combination and Justification

Recommendation: a model-based, goal-based agent (building a map as it explores) using IDS for planning when step costs are uniform; if different cells have different costs, use a utility-based (cost-aware) agent with UCS.

- The model-based component stores the explored map, which solves the unknown-layout and partial-observability problem.
- The goal-based component gives a clear objective (the exit) and a systematic way to plan towards it.
- IDS needs only O(b·d) memory, which suits a robot with limited onboard memory, and it still finds the shallowest path to the exit.
- UCS is chosen instead when moves have unequal costs, since only UCS guarantees the cheapest path, at the price of higher memory.

Thus, for a typical grid maze with equal step costs, a goal-based agent with a world model and IDS gives the best balance of completeness, optimality andmemory efficiency.

— End of Answers —